Planetary Gradient Removal - User Guide

Clément Violette / Yoships

Version 1.0

Planetary Gradient Removal (PGR) is a Python-based program made to remove bright background gradients of lucky imaging captures. It is aimed at solving the issue of bright evolving gradients causing erroneous frame quality estimations with softwares such as AS!4. It works using AVI and SER formats, with the ability to work on monochrome and RGB colour data.

Contents

1. Principle of gradient removal

Bright evolving gradients, such as the sky brightening during Sunrise, can be a problem with frame quality estimation in lucky imaging processes. Many softwares, such as the very used AS!4, use a local contrast-based process to estimate the quality of frames of a given dataset. But when a dataset contains a rapidly evolving gradient, it directly impacts the contrast in the image and leads to inconsistent quality estimations. The goal of Planetary Gradient Removal is to solve that issue.

The underlying problem is straightforward: the gradient is simply an additive signal superimposed on the planetary data, and can therefore be treated like any conventional deep-sky imaging gradient. The correction consists of estimating the intensity of this gradient independently for each frame, and subtracting it from that same frame :

$$Signal_{corrected} = Signal_{frame} - Gradient_{frame}$$

Because planetary imaging is usually carried out at very high focal length, the field of view is narrow enough that the gradient can be considered homogeneous across the frame and thus be represented by a single intensity value, one per colour channel for RGB data. To measure this per-frame intensity, the program asks the user, once, to select a background zone on a reference frame chosen to contain only background signal. For each frame, the mean value of the pixels within this zone is computed independently, giving the per-frame gradient, which is then subtracted from the frame. This operation is repeated for every frame of the sequence.

When several files are loaded as a batch, the same reference zone is reused, in pixel coordinates, across the batch. The program then outputs the gradient corrected video files, and their average gradients over the full sequence are exported as a 16-bit TIFF for reference.

2. User Guide

To start the program, you must execute it from a Python platform. Before executing, the video files need to be pre-aligned and debayered if in colour, using Planetary Imaging PreProcessor (PIPP) for example, and need to be at the same resolution/format. When executing it, a file explorator-type window will open to select either an .AVI or .SER file. If you select multiple files, the program will automatically set itself to batch mode.

Once a file is opened, the program will display a reference frame to select a zone containing only gradient. The program will then run through the batch, and outputs the gradient-corrected video files, as well as a reference mean gradient for each file and a log of the gradient's brightness across the video.

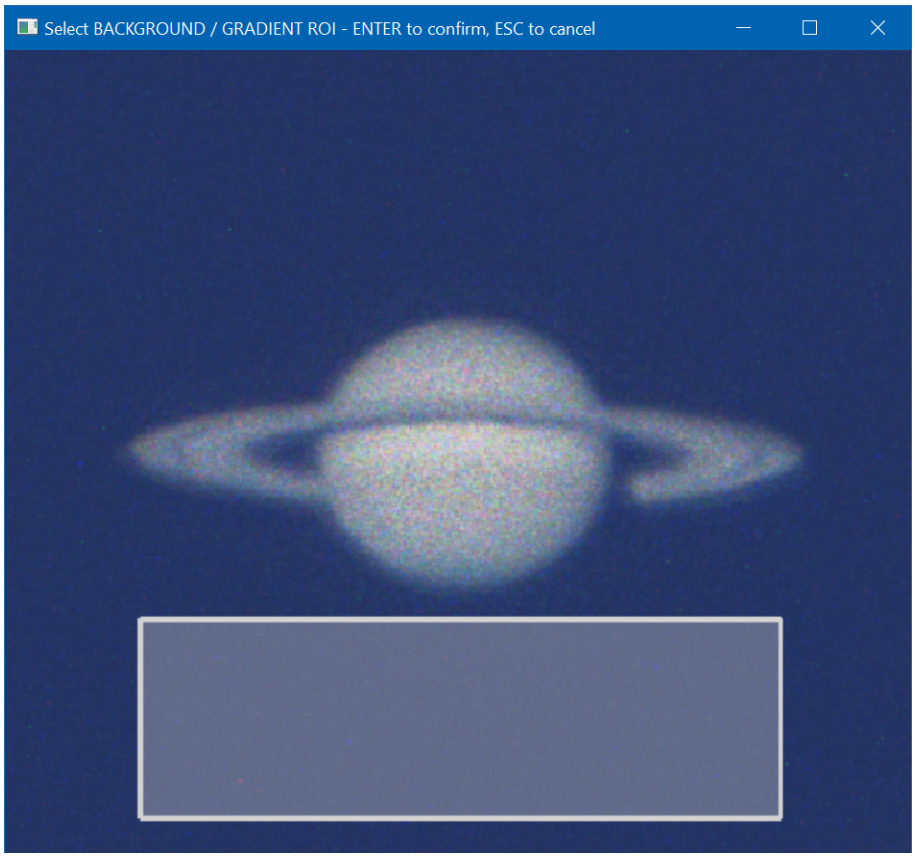

Figure 1: Example of a selection of gradient in a frame with Saturn during Sunrise.

Present in the head of the program are user parameters, such as gain and gamma for the output videos. The program can also normalize the gradient-corrected frames when `NORMALIZE_FRAMES` is set to `True` to a given percentage of saturation, under the variable name `NORMALIZE_TO_PERCENT`. However, i do not recommend normalizing the frames as it can interfere with the frame quality estimation of other softwares.

This program needs a few requirements to run efficiently. First, you need to install the Python libraries used in the code, it will not start without them. All the required installations are documented in the readme.txt file that is downloaded with the program.

This is the first version of this program, so it is not impossible that you will run into an error at some point. In case of a potential issue, you can report it to me on Discord, @Yoships, so it can be fixed in a future version.

Thanks for reading.